From Theory to Throughput:

Evolving Datalog's Semantics and Performance

Kristopher Micinski and HARP Lab

Introducing: The High-Performance Automated Reasoning and Programming (HARP) Lab

Collaboration of three PIs (Gilray, Kumar, Micinski) with many PhD/MS students since roughly 2019. We collaboratively built the systems I will describe; our students did significant amounts of work!

Yihao Sun
(SU PhD)

Chang Liu
(SU PhD)

Sowmith Kunapaneni,
(WSU PhD)

Ke Fan, UIC
(Grad: 2025!)

Ahmedur Rahman Shovon
(UIC PhD)

Arash Sahebolamri
(Grad: 2023!)

The HARP Lab

harp-lab.com

We are the High-performance Automated Reasoning and Programming (HARP) group.

Our research group is a joint collaboration between Washington State (Thomas Gilray), University of Illilnois at Chicago (Sidharth Kumar), and Syracuse (Kristopher Micinski). Collectively, we and our students build the next-generation of analytic and semantic reasoning engines to tackle large-scale challenges in program analysis, security, formal methods, knowledge representation, and medical reasoning.

Thomas Gilray

Washington State

Static Analysis, Programming Languages, HPC

Sidharth Kumar

U. Illinois at Chicago

HPC, Data Management, GPUs, Graph Analytics

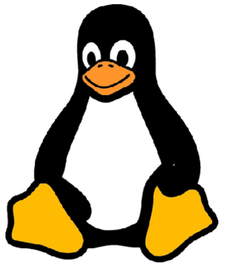
Kristopher Micinski

Syracuse University

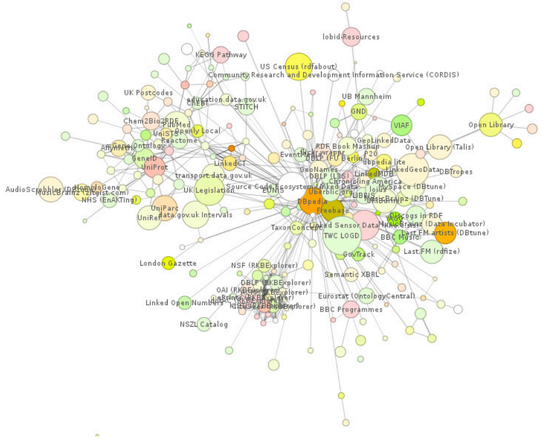
Programming Languages, Security, Automated Reasoning

Vision: take the world's hardest logical reasoning problems and scale them automatically on servers, GPUs, clusters, etc.

- Context-sensitive static analysis of Linux, httpd, …
- Reasoning over internet-scale graphs (RDFox, etc.)
- Drug discovery via reasoning over medical knowledge graphs



RDFOX



Soufflé

Apache

Specific to this talk… Our group has built the
fastest Datalog solvers currently available
on CPUs, GPUs, and supercomputers

- Why do we care? Datalog a **perfect fit** for many reasoning workloads
  - (Knowledge) graph queries (transitive closure, RDF, etc.)
  - Reachability-based abstract interpretation, finite-flow analysis
  - Superset disassembly, reverse engineering
  - Any recursive SQL-style workloads (semantic web querying, RDF, etc.)

Even putting aside the parallelism and scale, even ***implementing these applications correctly*** requires serious skill:

- Writing a program analysis in C++/Java/Racket/… is really tricky!
- Analysis implementation details form leaky abstractions
- The code (WALA, Soot, …) is very far away from the math

Imagine all of that ***plus*** wanting to scale your engine on a GPU cluster, running it on million-line programs, and continuously adding support for new high-performance hardware, abstractions, and analysis techniques...

Logic programming to the rescue?

Logic programming engines let you build your analysis (query, etc.) as a collection of rules in some logic

👍 — If your problem is a natural fit for the logic, you get awesome separation of concerns and a concisely-specified implementation
👍 — Modern engines use a host of techniques to achieve great scalability and parallelism, no need for system-specific optimization!
👎 — Might face an encoding-related (space/time) blowup if the logic doesn't perfectly match the application
👎 — A massively-parallel engine is useless if there's not enough available work

The Pitch

For the applications where Datalog is a natural fit, **use (the techniques from) an efficient Datalog engine**

If you don't, you'll just end up reinventing them anyway!

You focus on the rules, logical specifications, semantic correctness, etc.

We focus on…

- **Efficient compilation / evaluation**: semi-naïve, magic sets/demand-driven, rule scheduling, worst-case optimal joins, …
- **Automatic, massive data parallelism**: traditional shared-memory (CPU), SIMD/SIMT (GPUs), and distributed layouts
- **Index selection, data layout**: optimally-tuned data structures dovetail with compilation, cache / data locality, etc.

Three Contributions

I will discuss three specific contributions, each unique SOTA engines on…

- CPUs (Ascent—OOPSLA ’23, CC ’22): macro-embedded Datalog that integrates with user-provided tooling, the broader Rust ecosystem, with a focus on solving the *language integration problem*
- GPUs (ASPLOS ’25, AAAI ’25): Drill baby Drill (using HISA for GPU Datalog)
- Clusters (VLDB ’25, ICS ’25): Datalog∃!, Slog, and distributed Datalog

Datalog programs consist of a number of (definite, recursive) first-order implications

$$\text{Path}(x, y) \leftarrow \text{Edge}(x, y)$$

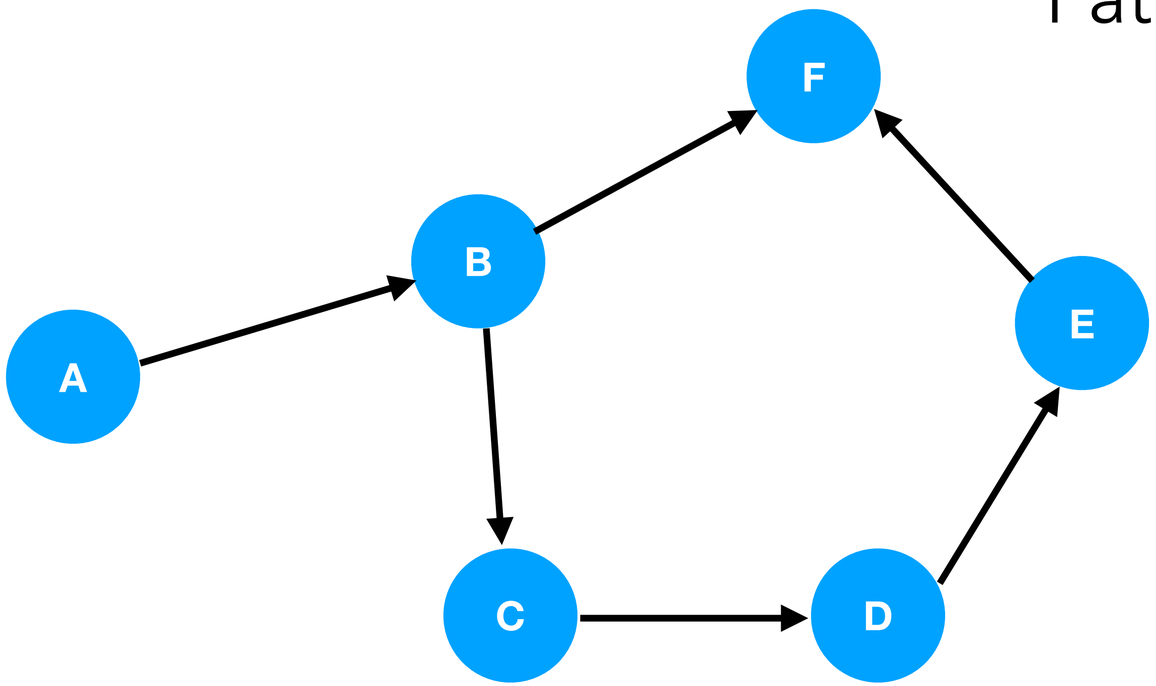
$$\text{Path}(x, z) \leftarrow \text{Edge}(x, y), \text{Path}(y, z)$$

Each rule is a definite Horn clause
(i.e., a clause with ***exactly one*** positive literal)

$$\text{Path}(x, y) \leftarrow \text{Edge}(x, y)$$
$$\equiv \text{Path}(x, y) \vee \neg\text{Edge}(x, y)$$
$$\text{Path}(x, z) \leftarrow \text{Edge}(x, y), \text{Path}(y, z)$$
$$\equiv \text{Path}(x, z) \vee \neg(\text{Edge}(x, y) \wedge \text{Path}(y, z))$$
$$\equiv \text{Path}(x, z) \vee \neg\text{Edge}(x, y) \vee \neg\text{Path}(y, z)$$

We also have **facts**, forming the input database
(the *extensional* database, i.e., **EDB**)

$$\text{Path}(x, y) \leftarrow \text{Edge}(x, y)$$
$$\text{Path}(x, z) \leftarrow \text{Edge}(x, y), \text{Path}(y, z)$$



Edge (EDB)

A	B
B	C
C	D
D	E
E	F
B	F

Solving Datalog: repeatedly apply rules

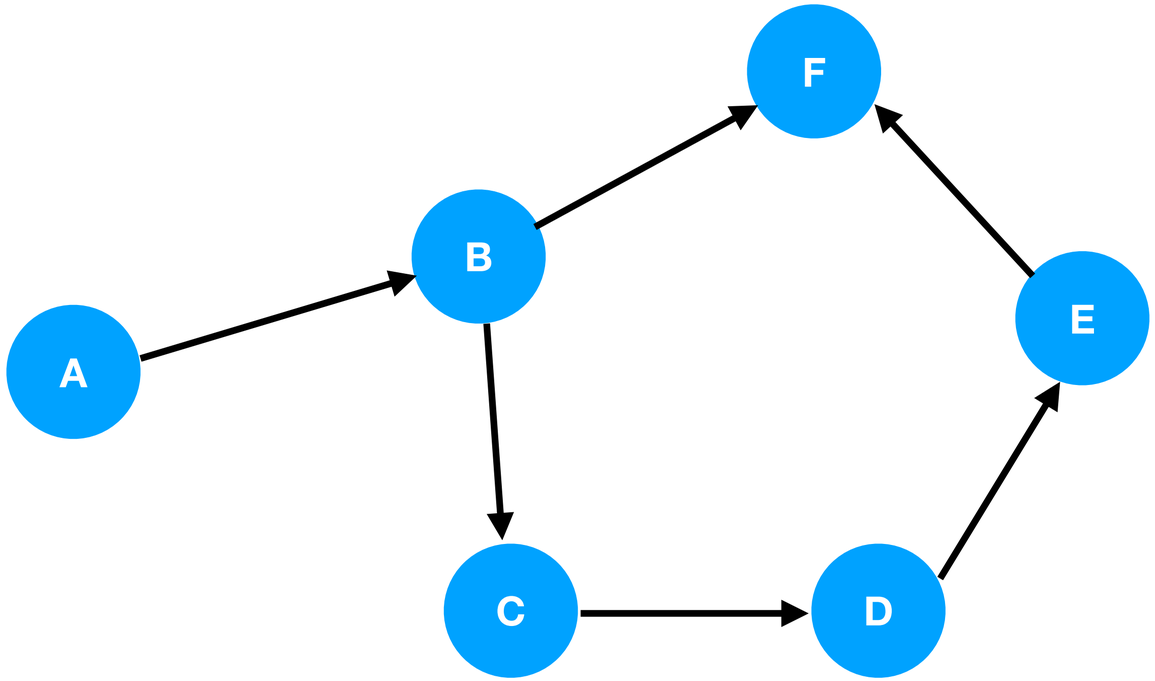
```
⟦P⟧ – Denotation of P, a set of rules
// Assume D is the set of ground constants in P and that P is safe
DB = {}
DB' = {}
Do:
  DB = DB'
  For each rule H(x,…) <- B0(y,…), … ∈ P:
    For every ground consequence you can find, add it to DB
Until DB' = DB
Return DB
```

Solving Datalog: repeatedly apply rules

```
⟦P⟧ — Denotation of P, a set of rules
// Assume D is the set of ground constants in P and that P is safe
DB = {}
DB′ = {}
Do:
  DB = DB'
  For each rule H(x,…) <- B0(y,…), … ∈ P:
    // For every ground consequence you can find, add it to DB
    // ∀θ, ground substitutions based on the current DB…
    For all substitutions θ such that codom(θ) ⊆ D and θ ⊨ body:
      DB′ = DB′ U θ(H(x,…)) // Apply θ to the head, add to DB
Until DB′ = DB
Return DB
```

Edge(x, y) ⟶

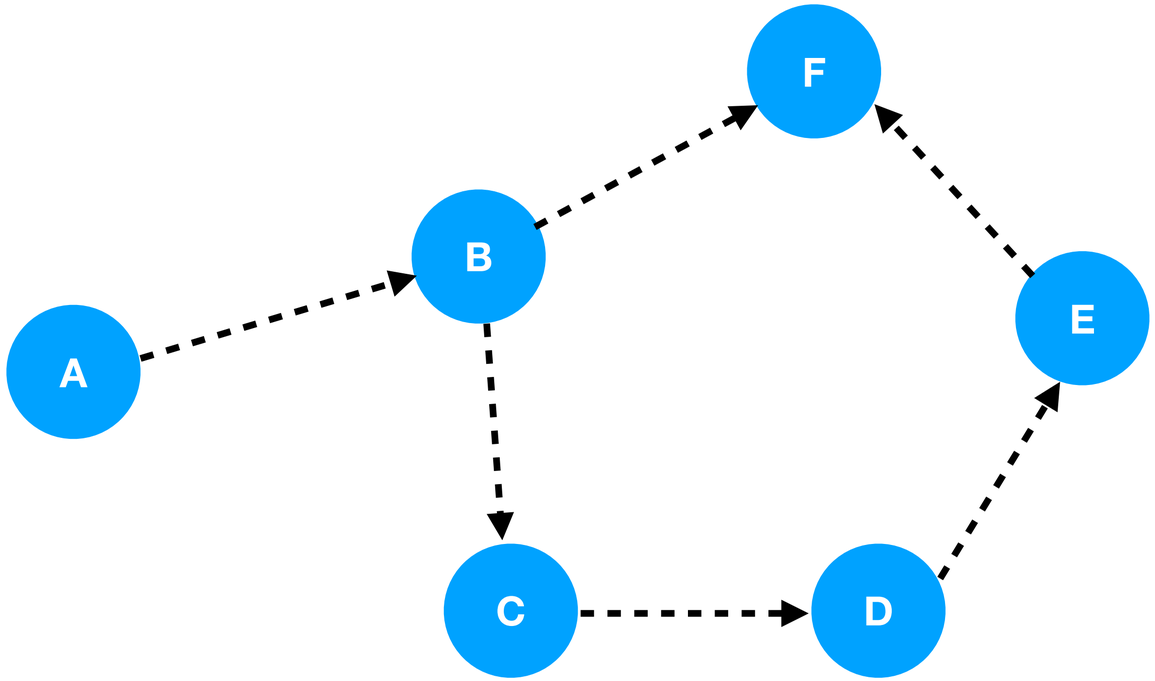
Path(x, y) ⇢



Edge(x, y) ⟶

Path(x, y) ⇢

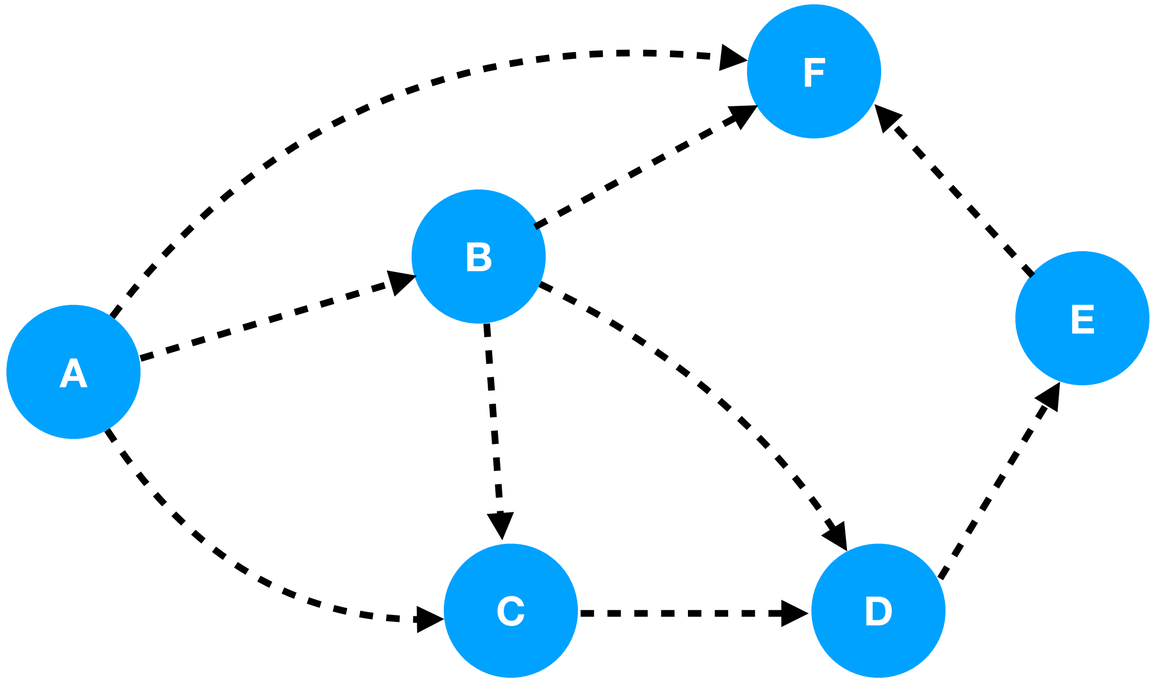
First, discover everything in Edge



Edge(x, y) ⟶

Path(x, y) ⇢

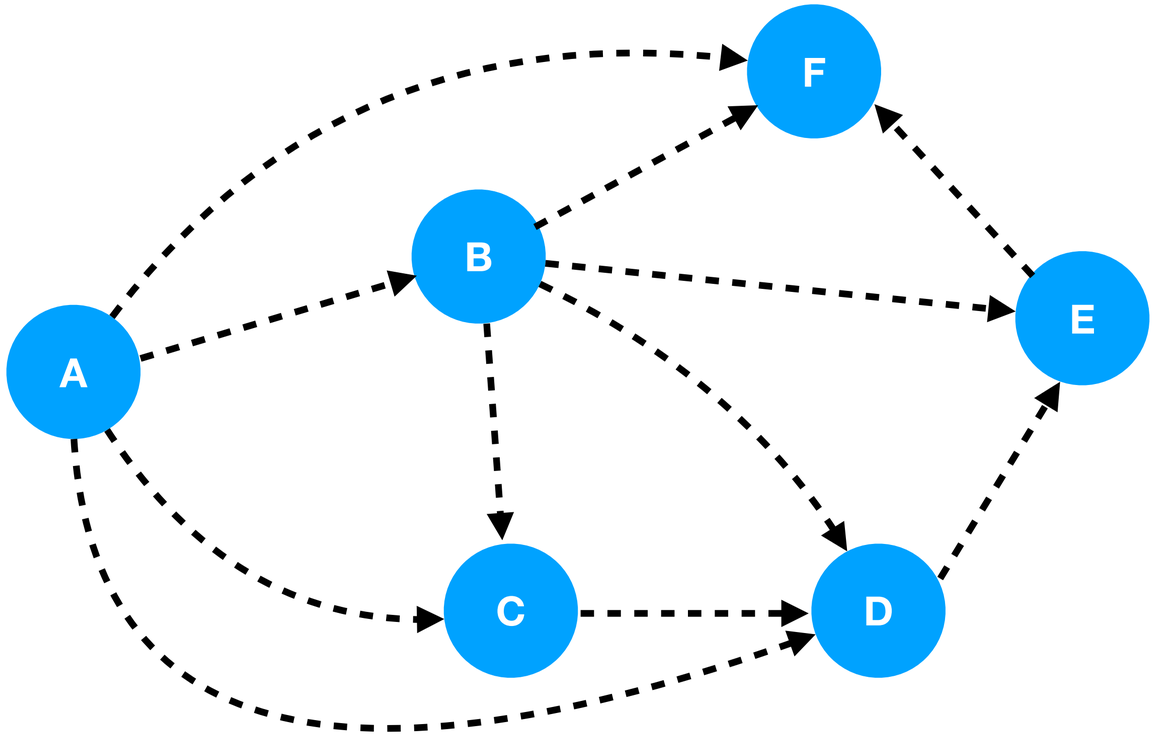
Next, ***rediscover*** everything previously in Path but **also** paths of length two



Edge(x, y) ⟶

Path(x, y) ⇢

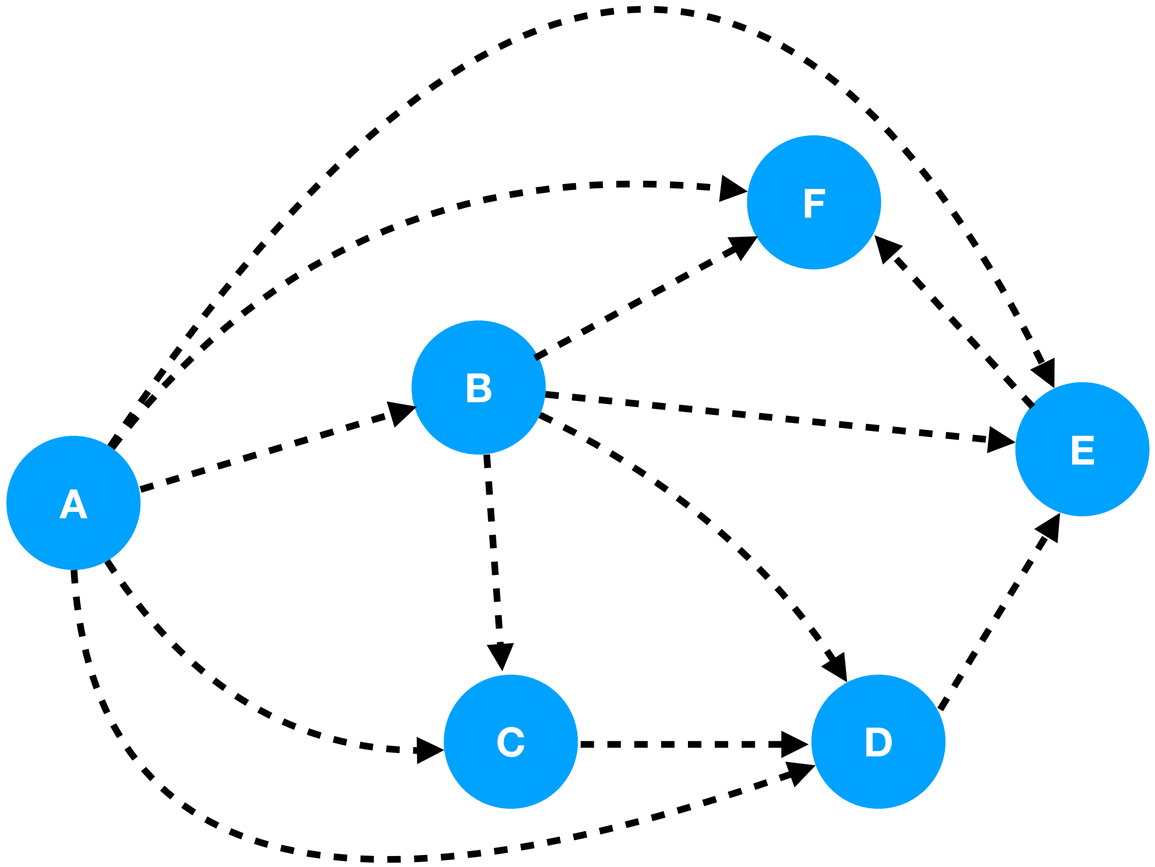
Next, rediscover and find paths up to length three



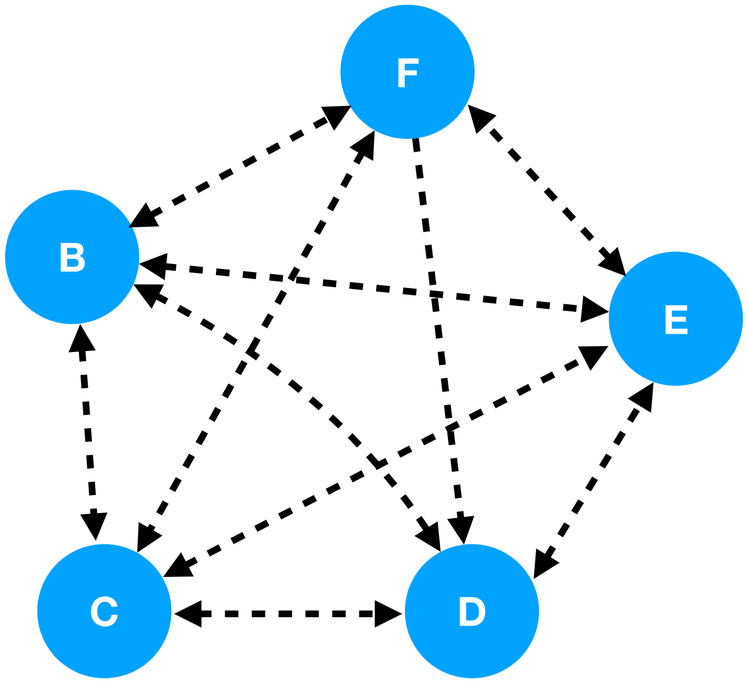
Edge(x, y) ⟶

Path(x, y) ⇢

Finally, up to length four—after which we're done



All (vanilla) Datalog programs terminate



The largest possible graph is a big clique of n nodes—
we’re in set semantics, so edges only count once

Cousot/Cousot fans will recognize this as abstract interpretation:

- The set of all possible ground tuples forms a lattice under ∪, ∩, ∅, …
- The application of rules to yield new ground tuples is monotonic
  - Tuples always added, never removed (see underlined line)
- Datalog termination is trivial consequence of fixed-point theorems
  - If set of atoms bounded, ⊤ is a finite set

```
⟦P⟧ — Denotation of P, a set of rules
// Assume D is the set of ground constants in P and that P is safe
DB = {}
DB′ = {}
Do:
  DB = DB'
  For each rule H(x,…) <- B0(y,…), … ∈ P:
    // ∀θ, ground substitutions based on the current DB…
    For all substitutions θ such that codom(θ) ⊆ D and θ ⊨ body:
      DB′ = DB′ U θ(H(x,…)) // Apply θ to the head, add to DB
Until DB′ = DB
Return DB
```

This algorithm is naïve

```
⟦P⟧ — Denotation of P, a set of rules
// Assume D is the set of ground constants in P and that P is safe
DB = {}
DB′ = {}
Do:
  DB = DB'
  For each rule H(x,…) <- B0(y,…), … ∈ P:
    // ∀θ, ground substitutions based on the current DB…
    For all substitutions θ such that codom(θ) ⊆ D and θ ⊨ body:
      DB′ = DB′ U θ(H(x,…)) // Apply θ to the head, add to DB
Until DB′ = DB
Return DB
```

At ***every*** iteration we *rediscover* everything we discovered from *every previous iteration*!

- The solution is called (in the literature) “semi-naïve evaluation:”
  - ***Really just a worklist algorithm!***
  - Because all functions are monotonic, we are pushing forward *positive diffs*
- This problem generalizes to “*incremental view maintenance*” (IVM)
  - Popular term in DB literature, various approaches

```
Input: P = rules + EDB facts (safe, positive)
DB  := ∅
Δ   := EDB

while Δ ≠ ∅:
  New := Newly-discovered tuples (exclude tuples in DB U Δ)
  DB  := DB U Δ
  Δ   := New

return DB
```

Logically, Datalog sits at an *interesting* place

- In some ways weak: no SAT, branching, and restricted to monotone theories
- Ehrenfeucht–Fraïssé games tell us **FO(LFP)** ⊃ **FO**!
  - FOL + fixed-point operator strictly more expressive!
  - Datalog ⊂ FO(LFP): FO(LFP) allows LFPs of any monotone operator definable in FOL
    - Datalog restricts us to the positive Horn (definite implication) fragment
- Weaker than SAT (i.e., existential second-order, ESO = NP)

$$\text{FO} (= \text{AC}^0) \subsetneq \text{Datalog} \subsetneq \text{FO(LFP)} \subseteq \text{ESO} (= \text{NP})$$

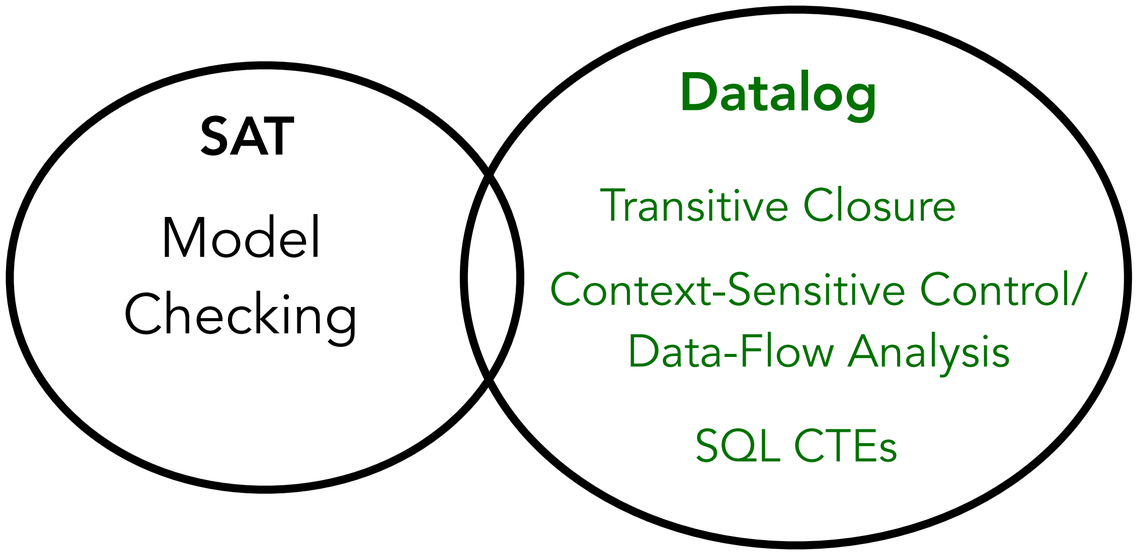
(Descriptive Complexity: FO over ordered, finite structures)

Canonical “Branching” Problem

(EXPTIME, requires exploring ever-increasing branching combinations)

Canonical “Recursive” Problem

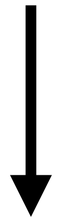
(PSPACE-complete, bounded by number of input atoms)



Compilation to RA kernels

- Compile each rule into relational algebra (an optimized join plan):

$$T(x, z) :- R(x, y), S(y, z), y > 3, x \neq z.$$



$$T = \pi_{R.x,S.z} (\sigma_{R.y>3 \wedge R.x \neq S.z} (R \bowtie_{R.y=S.y} S))$$

Join, then filter, then project

Executing RA kernels

Multiple options here:

- Write an interpreter: ~easier to change evaluation strategy (“tuple at a time,”), easier to build debugging infrastructure, etc.

- Compile: identify the tight loop of the interpreter and just emit that code. Crucially: avoid per-iteration / per-tuple work
  - May be easier to get cache locality
  - Generally faster, used in most high-perf implementations
  - Can ***fuse*** join/project/select/filter, end up with join+X kernels

Operationalizing Joins the Easy Way

- Just use binary joins, binary joins make sense, etc.
- A(x,y) ⋈ E(x,z) is operationalized as…
  - “Scan A to iterate pairs (x,y), range-select E to get zs for x.”
  - Needs fast range querying

Still some trade-offs:

```
OrderInfo(o, c, p, d) :-
    Orders(o, c, p, d),
    Customer(c, region),
    Product(p, category).
```

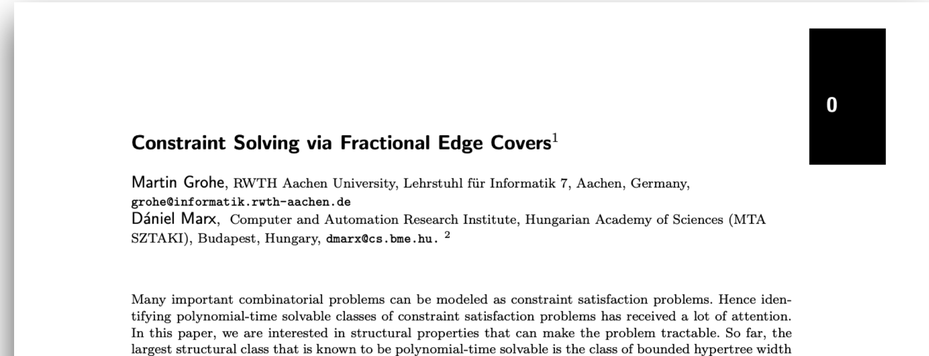
- How do I partition this into binary joins?
- Order matters: use dynamic data?
- Do I *materialize* intermediate outputs?

Soufflé/Ascent use pipelined (don’t materialize intermediates) binary hash joins

The Bad Joins

- The AGM bound tells us that for some queries, we are ***destined to failure*** if we use a binary join plan
- For *cyclic* queries, binary joins generate super-linear intermediate results (in time / space)

  Triangle(x,y,z) :- E(x,y), E(y,z), E(z,x).
- No *efficient* way to do this with binary joins!



0

Constraint Solving via Fractional Edge Covers[1]

Martin Grohe, RWTH Aachen University, Lehrstuhl für Informatik 7, Aachen, Germany, grohe@informatik.rwth-aachen.de
Dániel Marx, Computer and Automation Research Institute, Hungarian Academy of Sciences (MTA SZTAKI), Budapest, Hungary, dmarx@cs.bme.hu. [2]

Many important combinatorial problems can be modeled as constraint satisfaction problems. Hence identifying polynomial-time solvable classes of constraint satisfaction problems has received a lot of attention. In this paper, we are interested in structural properties that can make the problem tractable. So far, the largest structural class that is known to be polynomial-time solvable is the class of bounded hypertree width

Operationalizing Joins the Hard Way

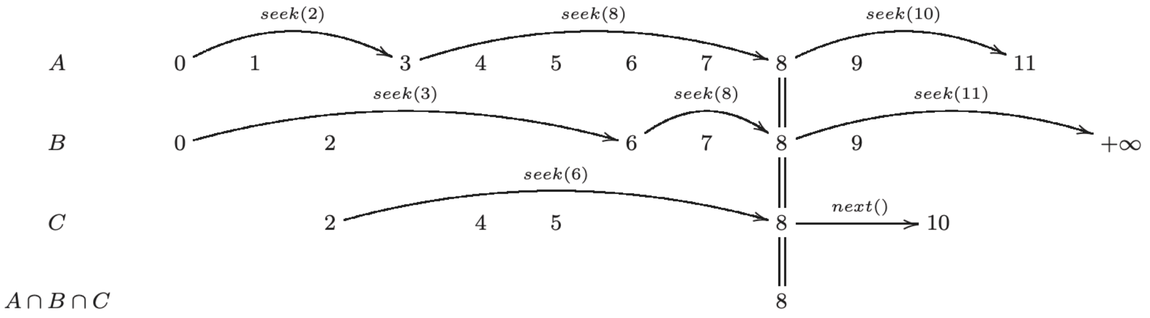
Triangle(x,y,z) :- E(x,y), E(y,z), E(z,x).

Various ***worst-case optimal*** joins exist:

- You cannot materialize the intermediate product in time/space
- Instead of "join at a time," these do "tuple at a time,"
- Need fast multi-way set intersection, e.g., using trie iterators

Leapfrog Triejoin: A Simple, Worst-Case Optimal Join Algorithm

Todd L. Veldhuizen
LogicBlox Inc.
Two Midtown Plaza
1349 West Peachtree Street NW
Suite 1880, Atlanta GA 30309
tveldhui@{logicblox.com,acm.org}



How do you store relations / tuples?

Need a good relation-backing DS, needs coordination w/ joins

Soufflé, Ascent, many other traditional Datalogs:

- Use BTreeMap, HashMap, etc.
  - Key on the index, map to a set of rows (i.e., full tuples)
  - How to handle multiple indices? Duplication
  - Want fast parallel scan, minimal locking

```
path(x,y), edge(y,z)

R(z), edge(x,z)
```

Optimizing Tuple Storage

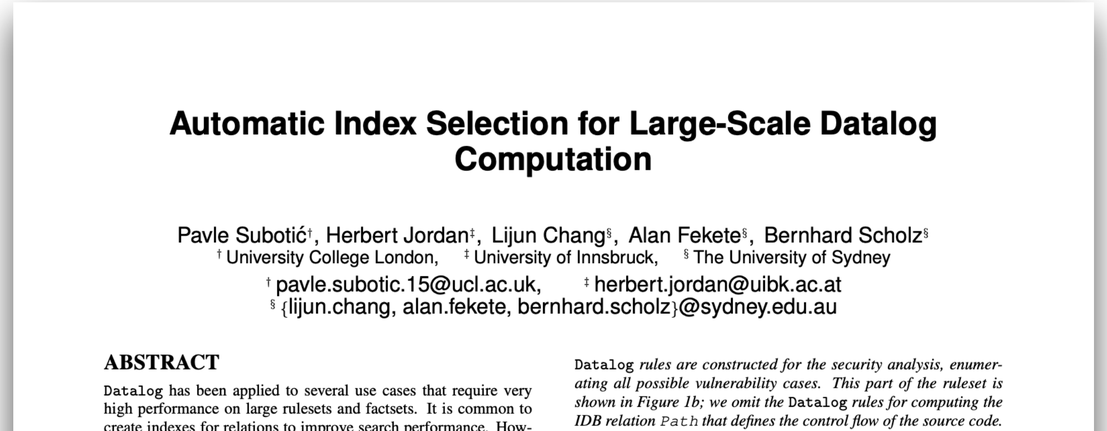
Duplicating a huge table (for each index) is a real bummer

In some cases we can do better: `E(x,y), R(x,y,z)`

`G(x), R(x,y,z)`

Here, we can organize R as a *trie* which puts x first, then y, then z: so the index {x} is the (trie) prefix of the index {x,y}

Don't always want to use tries!



Automatic Index Selection for Large-Scale Datalog Computation

Pavle Subotić[†], Herbert Jordan[‡], Lijun Chang[§], Alan Fekete[§], Bernhard Scholz[§]
[†] University College London, [‡] University of Innsbruck, [§] The University of Sydney
[†] pavle.subotic.15@ucl.ac.uk, [‡] herbert.jordan@uibk.ac.at
[§] {lijun.chang, alan.fekete, bernhard.scholz}@sydney.edu.au

ABSTRACT

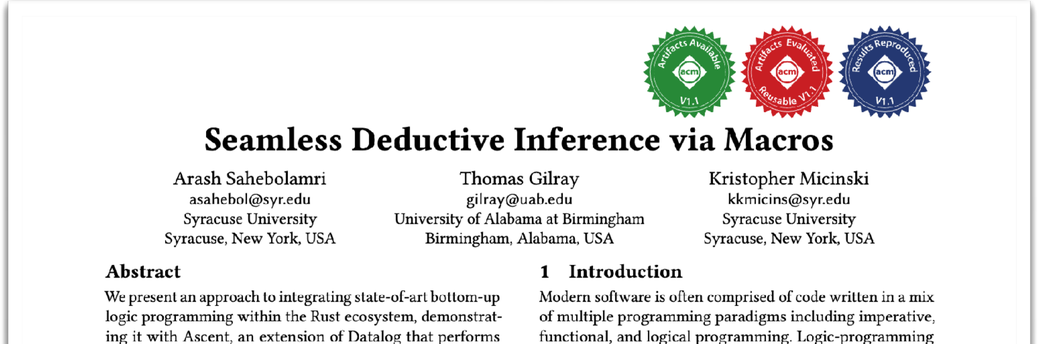
Datalog has been applied to several use cases that require very high performance on large rulesets and factsets. It is common to create indexes for relations to improve search performance. How-

Datalog *rules are constructed for the security analysis, enumerating all possible vulnerability cases. This part of the ruleset is shown in Figure 1b; we omit the* Datalog *rules for computing the IDB relation* Path *that defines the control flow of the source code.*

Ascent, a fast, Rust-embedded Datalog

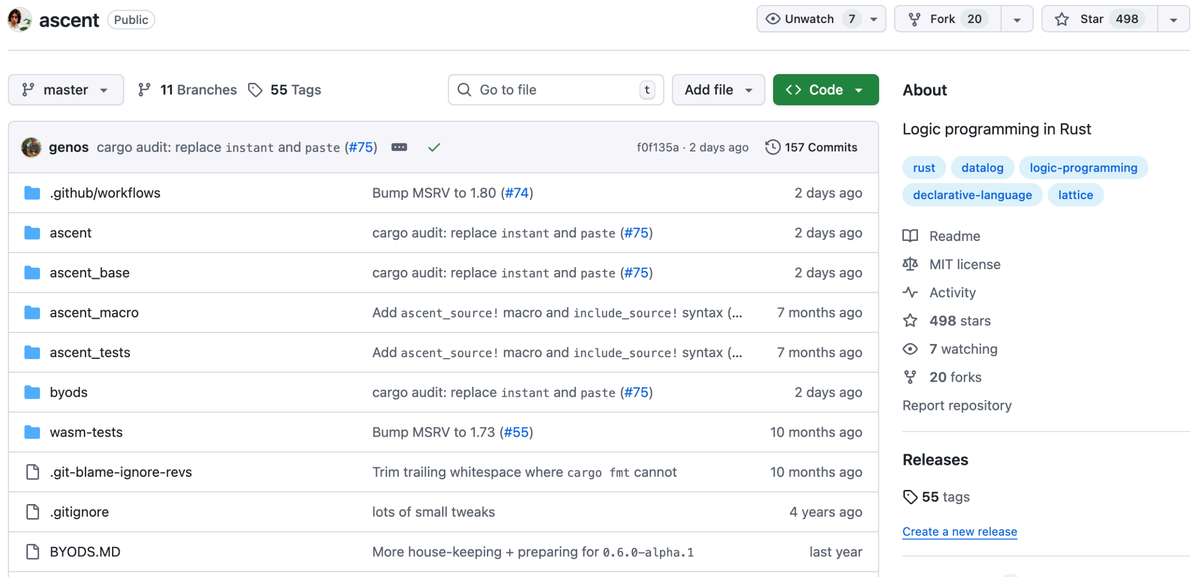
- A macro-based Datalog engine embedded as Rust macros.
- Performance competitive with or faster than Soufflé (~SOTA, good baseline)
- Better parallelism / scalability via Rayon (scales up as far as we've tested, ~64threads)
- Nicely integrates with Rust's data structures
  - Writing an analysis of LLVM? Don't manually flatten to CSV, just use the llvm crate!
- Also includes the Bring Your Own Data Structures (BYODS) approach
  - Enables writing "relation providers" for more compressed data structures

`https://s-arash.github.io/ascent/`



Where does Ascent really win?

- “I want to use Datalog within the context of my giant Rust program, but I don’t want to have to write out a ton of .CSVs, or link in a binary blob with an opaque API.”
- The usability story is excellent: arbitrarily mix Rust / Ascent code
- An engineer at Galois was able to use Ascent to write yapall, a 1.5kloc abstract interpreter in Ascent using standard Rust tooling!




```
use ascent::ascent;
ascent! {
    relation edge(i32, i32);
    relation path(i32, i32);
    path(x, y) <-- edge(x, y);
    path(x, z) <-- edge(x, y), path(y, z);
}
fn main() {
    let mut prog = AscentProgram::default();
    prog.edge = vec![(1, 2), (2, 3)];
    prog.run();
    println!("path: {:?}", prog.path);
}
```

yapall

- Yet Another Pointer Analysis for LLVM, written in Ascent (~1.5kloc Ascent code)
- Context-sensitive points-to analysis of LLVM code, with tunable precision
- Follows the abstracting-abstract machines style

Prog.	k	Pts	Time (s) by thread count								Memory (MiB)	
			8		16		32		64			
			ind	def	ind	def	ind	def	ind	def	ind	def
Jackson	3	10.2M	8.8	10.9	7.6	10.8	7.4	10.1	7.3	9.7	1,940	3,107
	4	164M	128	166	102	164	99	152	94	143	30,466	50,533
Luac	0	3.2M	5.2	4.9	3.8	4.4	3.2	4.4	3.3	5.0	555	732
	1	45M	68	58	48	52	45	50	52	50	3,798	8,000
Lua	0	12.8M	19.6	16.5	13.0	14.6	10.4	14.6	9.7	15.8	1,782	3,222
	1	214M	303	257	193	215	154	199	183	201	14,074	34,335
httpd	0	27.1M	154	103	91	72	65	59	58	61	3,545	6,100
	1	5.39B	26,530	OOM	15,240	–	10,285	–	9,793	–	425,000	OOM
SQLite	0	167M	830	540	471	367	312	284	248	326	26,665	44,597
Redis	0	735M	19,083	11,536	9,210	6,486	5,424	4,087	4,063	3,541	99,500	178,264

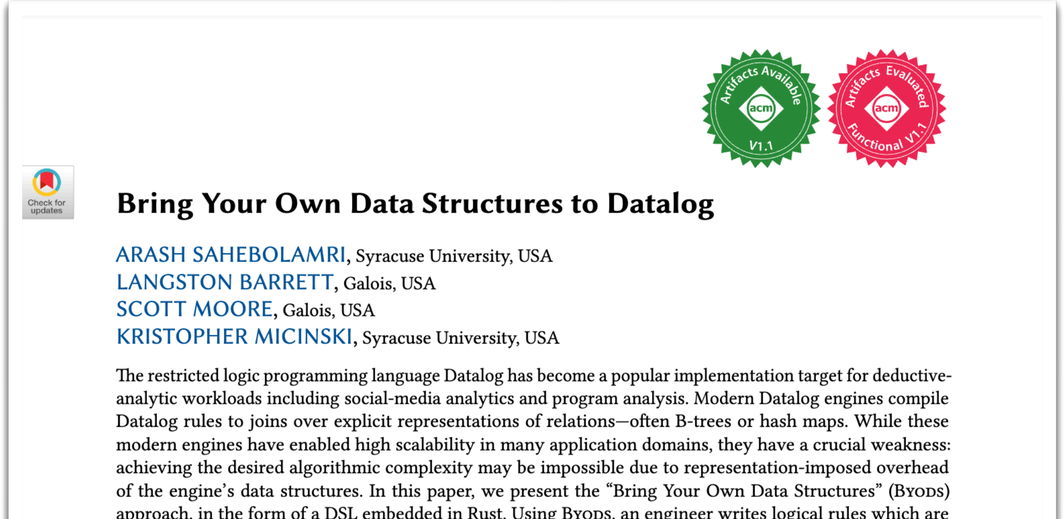
Bring Your Own Data Structures

- OOPSLA ’23 paper: what if you want to use arbitrary Rust datatypes as relation-backing tuple stores?
- Advantages: can exploit application-specific characteristics to compress relations
- Object protocol exports an interface to communicate back-and-forth with Ascent
- You provide range querying (for an index), scanning, etc.
- Still binary joins, however (no WCOJ)

```
use ascent_byods_rels::trrel_uf;
ascent! {
   relation edge(Node, Node);

   #[ds(trrel_uf)] // Makes the relation transitive
   relation path(Node, Node);

   path(x, y) <-- edge(x, y);
}
```



Artifacts Available acm V1.1

Artifacts Evaluated acm Functional V1.1

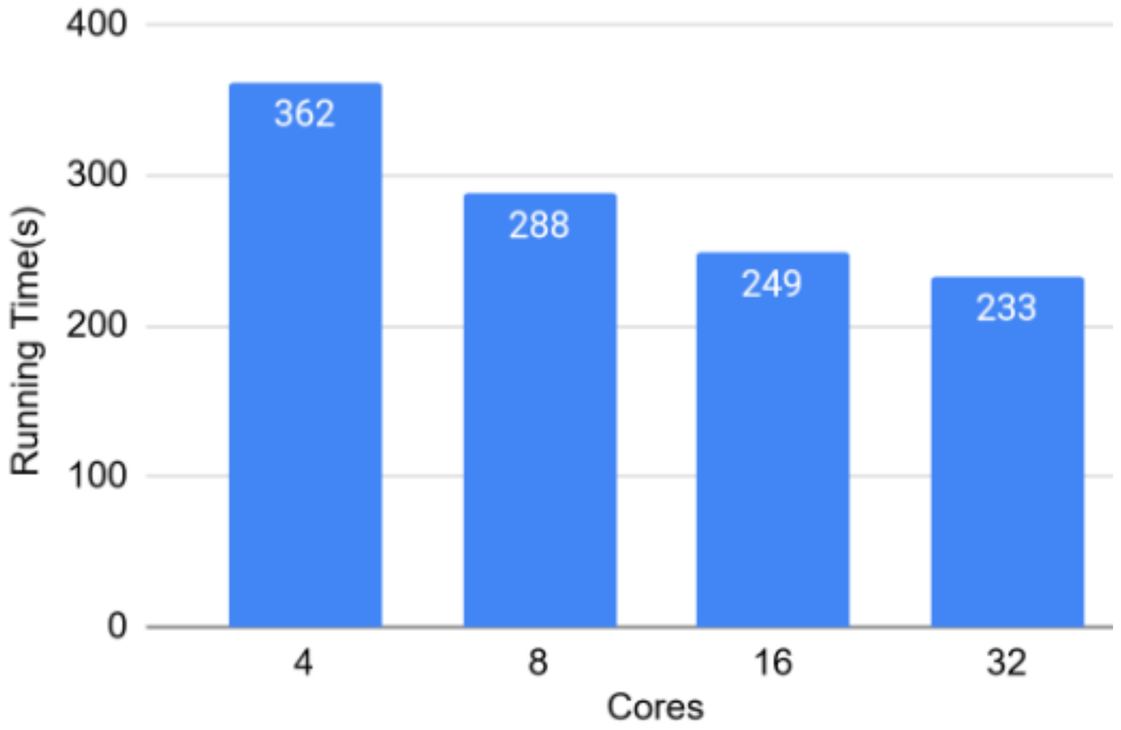
Check for updates

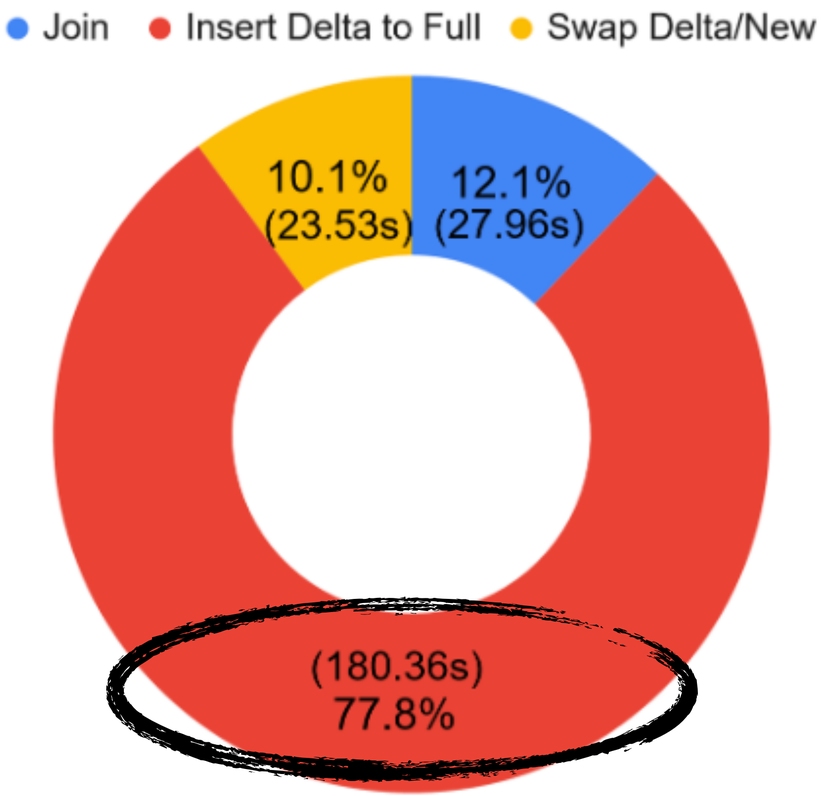
Bring Your Own Data Structures to Datalog

ARASH SAHEBOLAMRI, Syracuse University, USA
LANGSTON BARRETT, Galois, USA
SCOTT MOORE, Galois, USA
KRISTOPHER MICINSKI, Syracuse University, USA

The restricted logic programming language Datalog has become a popular implementation target for deductive-analytic workloads including social-media analytics and program analysis. Modern Datalog engines compile Datalog rules to joins over explicit representations of relations—often B-trees or hash maps. While these modern engines have enabled high scalability in many application domains, they have a crucial weakness: achieving the desired algorithmic complexity may be impossible due to representation-imposed overhead of the engine’s data structures. In this paper, we present the “Bring Your Own Data Structures” (BYODS) approach, in the form of a DSL embedded in Rust. Using BYODS, an engineer writes logical rules which are

- CPU-based engines do not scale beyond ~8-16 cores
- Serialized tuple insertion/deduplication is a huge issue
  - *77.8% of runtime in Soufflé at 16 cores on TC!*

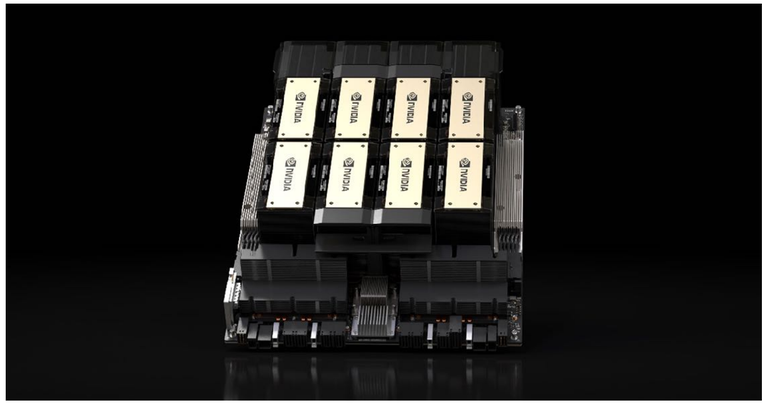




We *want* GPU Datalog!

- Modern GPUs: massive parallelism
- ***Extreme*** memory bandwidth
  - H100 SXM — 3.35 TB/s
  - AMD Instinct MI325X — 6 TB/s!
- SIM**T** architecture
  - No overhead from inter-warp divergence!

Unfortunately, no current GPU Datalog truly beats Soufflé on a single CPU!

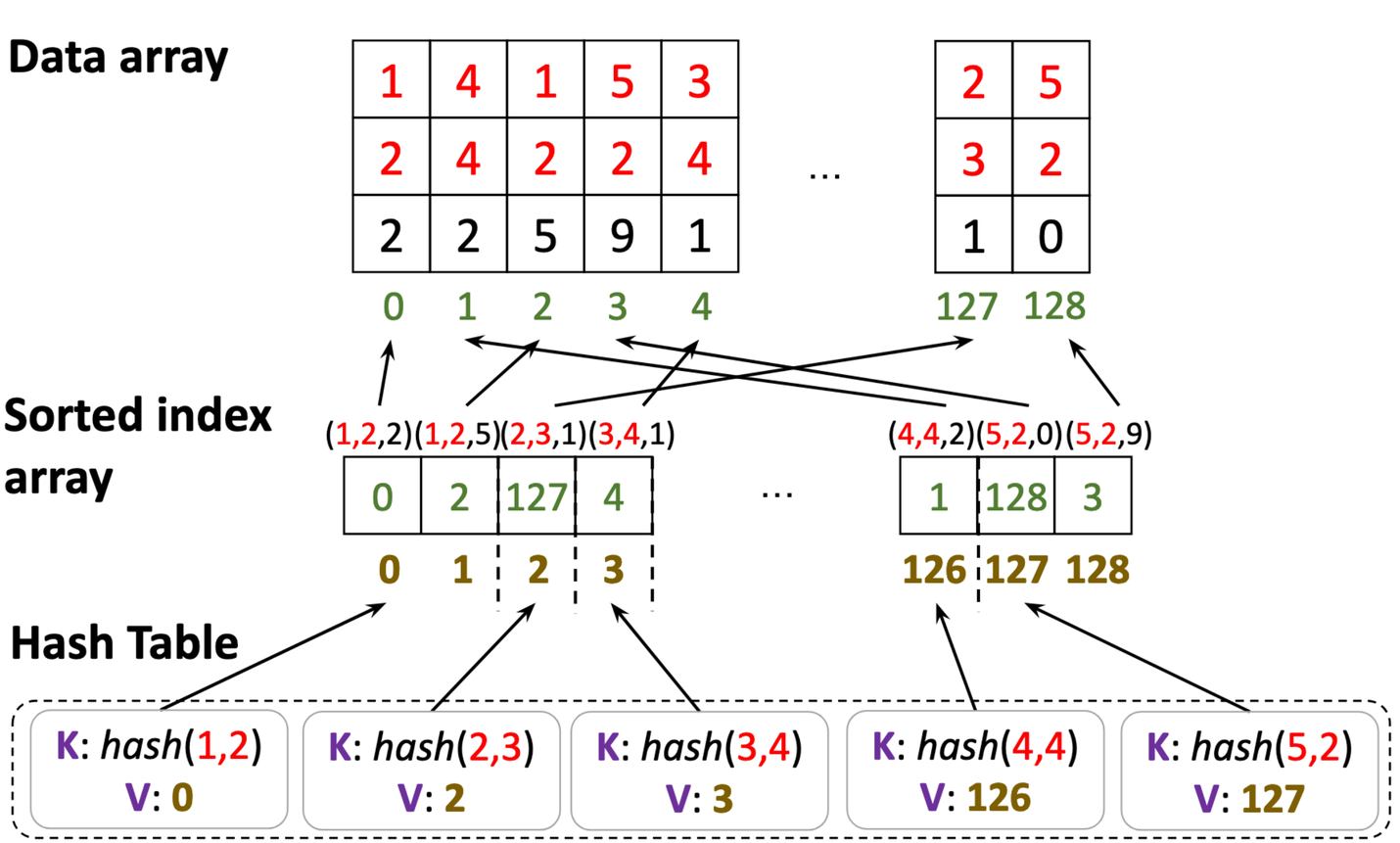


Contribution: **H**ash-**I**ndexed **S**orted **A**rray (HISA)

Scanning

Deduplication

Range querying



GDLog (ASPLOS ’25) – the first *fast* GPU Datalog

- HISA facilitates implementing fast iterated RA kernels
- GDLog — Our Datalog engine using HISA
- Most RA kernels straightforward, but…
- Joins on a HISA decomposed in two phases:
  - (1) Join-cardinality estimation / allocation
  - (2) Materialization
- Followed by deduplication, iteration, fixpoint check

Evaluation

- GDLog vs. SOTA competitors on GPUs and CPUs
- We compare against Soufflé, GPUJoin, and Soufflé
- Systems:
  - 1 Nvidia H100 80G + 1 AMD EPYC 7713
  - 1 Nvidia A100 40G + 1 AMD EPYC 7543 (ANLF's Polaris)
  - 1 AMD MI250 64G + 1 AMD EPYC 7713

Same Generation (SG): GDlog vs. Soufflé (32 threads) and cuDF

Dataset	SG size	Time (s)			
		GDlog	GDlog-HIP	Soufflé	cuDF
fe_body	408M	**5.05**	19.57	74.26	OOM
loc-Brightkite	92.3M	**3.42**	14.00	48.18	OOM
fe_sphere	205M	**2.36**	8.48	48.12	OOM
SF.cedges	382M	**5.54**	20.57	68.88	OOM
CA-HepTH	74M	**2.79**	5.92	20.12	21.24
ego-Facebook	15M	**1.23**	2.81	17.01	19.07

- GDlog always fastest
- Generally ~10x vs. others
- Nvidia is better than AMD (memory allocation)

Transitive Closure vs. Soufflé, GPUJoin, and cuDF

Dataset name	*Reach* edges	Time (s) GDLog	Soufflé	GPUJoin	cuDF
com-dblp	1.91B	**14.30**	232.99	OOM	OOM
fe_ocean	1.67B	**23.36**	292.15	100.30	OOM
vsp_finan	910M	**21.91**	239.33	125.94	OOM
Gnutella31	884M	**5.58**	96.82	OOM	OOM
fe_body	156M	**3.76**	23.40	22.35	OOM
SF.cedge	80M	**1.63**	33.27	3.76	64.29

Context-Sensitive Points-To Analysis

$$
\begin{aligned}
ValueFlow(x, y) &\leftarrow ValueFlow(x, z), ValueFlow(z, y).\\
ValueAlias(x, y) &\leftarrow ValueFlow(z, x), ValueFlow(z, y).\\
ValueFlow(x, y) &\leftarrow assign(x, z), MemoryAlias(z, y).\\
MemoryAlias(x, w) &\leftarrow dereference(y, x), ValueAlias(y, z),\\
&\quad\ dereference(z, w).\\
ValueAlias(x, y) &\leftarrow ValueFlow(z, x), MemoryAlias(z, w),\\
&\quad\ ValueFlow(w, y).\\
ValueFlow(y, x) &\leftarrow assign(y, x).\\
ValueFlow(x, x) &\leftarrow assign(x, y).\\
ValueFlow(x, x) &\leftarrow assign(y, x).\\
MemoryAlias(x, x) &\leftarrow assign(y, x).\\
MemoryAlias(x, x) &\leftarrow assign(x, y).
\end{aligned}
$$

Andersen-style pointer analysis implemented as Datalog rules

"Graspan: A Single-machine Disk-based Graph System for Interprocedural Static Analyses of Large-scale Systems Code"
[ASPLOS '17]

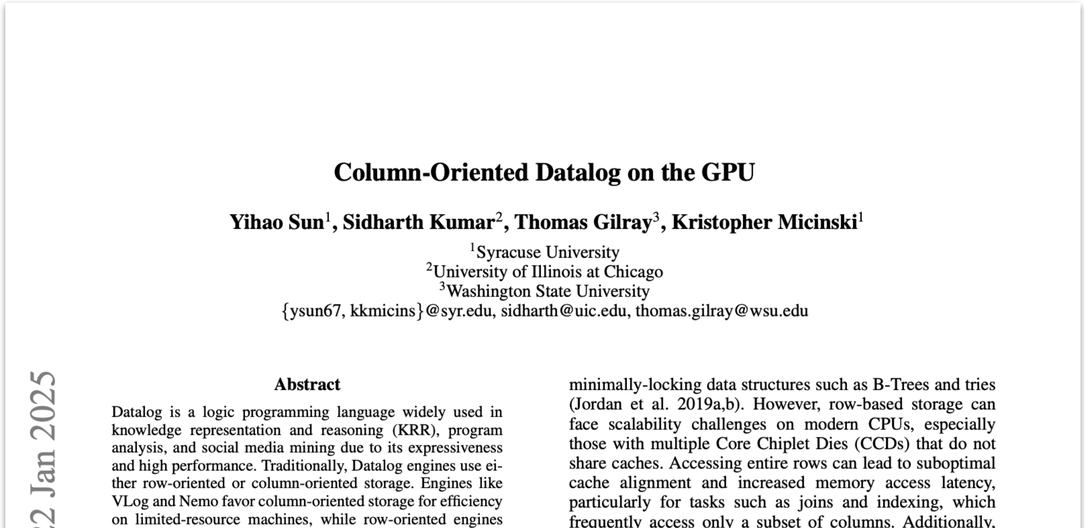
CSPA: GDLog vs. Soufflé

Dataset Name	Input Relation Size	Output Relation Size	Time (s) GDLog	Time (s) Soufflé	Speedup
Httpd	Assign: 3.62e5 Dereference: 1.14e6	ValueFlow: 1.36e6 ValueAlias: 2.34e8 MemoryAlias:8.89e7	**1.33**	49.48	**37.2x**
Linux	Assign: 1.98e6 Dereference: 7.50e6	ValueFlow: 5.50e6 ValueAlias: 2.23e7 MemoryAlias:8.84e7	**0.39**	13.44	**34.5x**
PostgreSQL	Assign: 1.20e6 Dereference: 3.46e6	ValueFlow: 3.71e6 ValueAlias: 2.23e8 MemoryAlias:8.84e7	**1.27**	57.82	**44.9x**

GDLog (NVIDIA H100) vs. Soufflé (EPYC 7543P on Polaris)

Column-Oriented Datalog on the GPU

- AAAI ’25: instead of grouping rows together, group *columns* together
- DuckDB and similar engines: very popular on the CPU
- Lots of potential advantages: cache friendly, coalesced writes, etc.
- We found it was a huge win: up to 200x vs. optimally-tuned Soufflé
  - Faster server chip w/ 64 physical cores vs. H100



Column-Oriented Datalog on the GPU

Yihao Sun[1], Sidharth Kumar[2], Thomas Gilray[3], Kristopher Micinski[1]
[1]Syracuse University
[2]University of Illinois at Chicago
[3]Washington State University
{ysun67, kkmicins}@syr.edu, sidharth@uic.edu, thomas.gilray@wsu.edu

Abstract

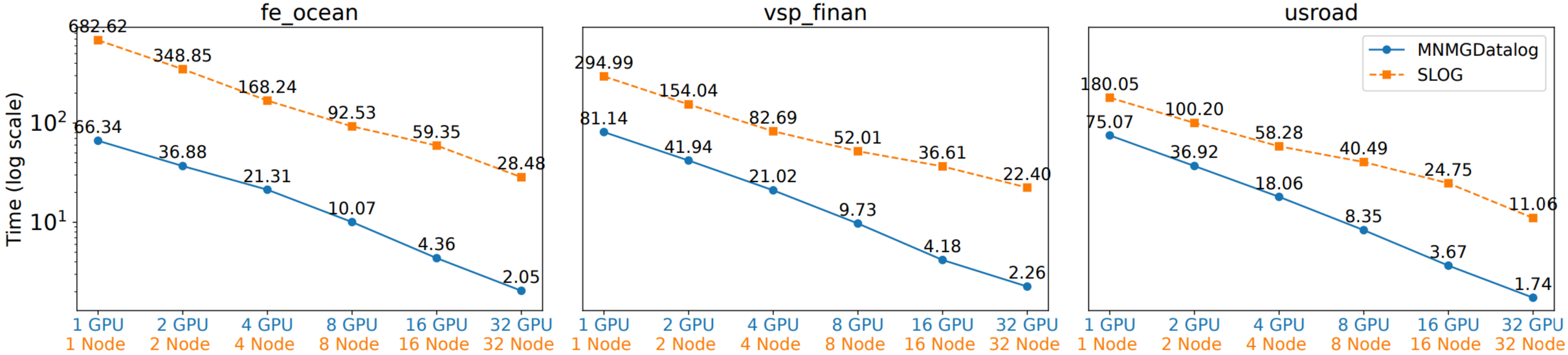
Datalog is a logic programming language widely used in knowledge representation and reasoning (KRR), program analysis, and social media mining due to its expressiveness and high performance. Traditionally, Datalog engines use either row-oriented or column-oriented storage. Engines like VLog and Nemo favor column-oriented storage for efficiency on limited-resource machines, while row-oriented engines minimally-locking data structures such as B-Trees and tries (Jordan et al. 2019a,b). However, row-based storage can face scalability challenges on modern CPUs, especially those with multiple Core Chiplet Dies (CCDs) that do not share caches. Accessing entire rows can lead to suboptimal cache alignment and increased memory access latency, particularly for tasks such as joins and indexing, which frequently access only a subset of columns. Additionally,

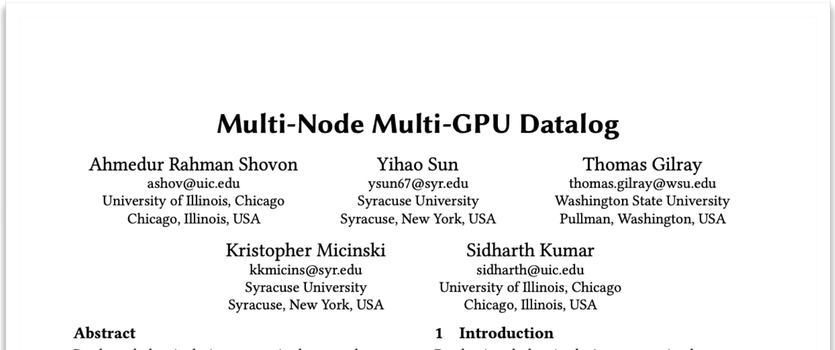
Performance on a set of standard DB benchmarks (LUBM).
FVLog(C) is our system on the *CPU* and (G) is on an H100

Dataset	FVLOG(C)	FVLOG(G)	Nemo	VLog	RDFox
010	0.15	0.01	0.65	1.46	0.44
100	0.71	0.03	6.34	5.02	4.98
01K	6.47	0.16	62.32	165.8	56.8

ICS ’25: **Multi-Node Multi-GPU** Datalog

Strong scaling: transitive closure (1 GPU/node)





Multi-Node Multi-GPU Datalog

Ahmedur Rahman Shovon
ashov@uic.edu
University of Illinois, Chicago
Chicago, Illinois, USA

Yihao Sun
ysun67@syr.edu
Syracuse University
Syracuse, New York, USA

Thomas Gilray
thomas.gilray@wsu.edu
Washington State University
Pullman, Washington, USA

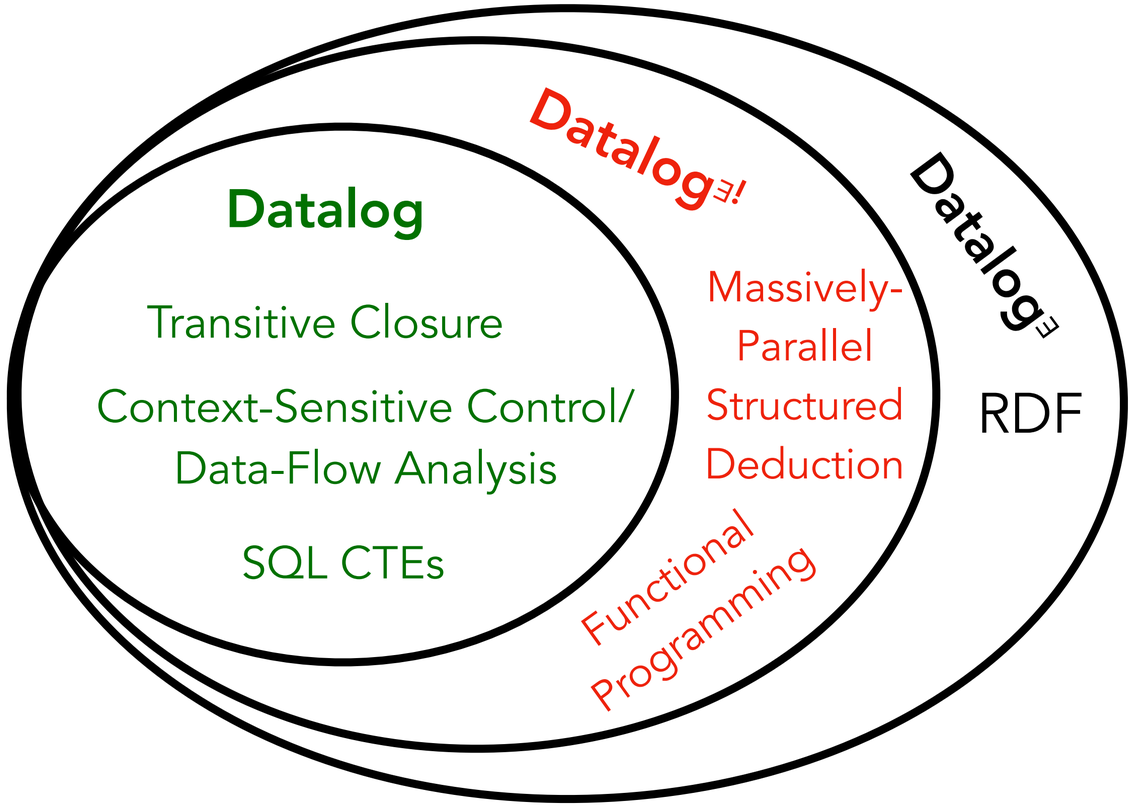
Kristopher Micinski
kkmicins@syr.edu
Syracuse University
Syracuse, New York, USA

Sidharth Kumar
sidharth@uic.edu
University of Illinois, Chicago
Chicago, Illinois, USA

Abstract

1 Introduction

Next, I'd like to start talking about Slog, our extension of Datalog to **first-class facts** (S-expressions)



Declarative Program Analysis

`Assign(x, y)`
stores syntactic expressions
`x = y;`

`Dereference(x, y)`
stores syntactic expressions
`x = *y;`

A target of analysis is encoded extensionally as input relations.

Syntax-directed evaluation rules deduce control-flow, data-flow, alias, and other semantic information about the program.

Declarative Program Analysis

`Assign(x, y)`
stores syntactic expressions
`x = y;`

`Dereference(x, y)`
stores syntactic expressions
`x = *y;`

```
// Immediate aliases and data flows
ValueFlow(y, x) :- Assign(y, x).
ValueFlow(x, x) :- Assign(x, y).
ValueFlow(x, x) :- Assign(y, x).
MemoryAlias(x, x) :- Assign(y, x).
MemoryAlias(x, x) :- Assign(x, y).
```

```
// Transitive flows (a TC computation)
ValueFlow(x, y) :- ValueFlow(x, z),
                   ValueFlow(z, y).
```

```
// Deduced may alias and alias-based data-flow information
ValueAlias(x, y) :- ValueFlow(z, x), ValueFlow(z, y).
MemoryAlias(x, w) :- Dereference(y, x), ValueAlias(y, z), Dereference(z, w).
ValueAlias(x, y) :- ValueFlow(z, x), MemoryAlias(z, w), ValueFlow(w, y).
ValueFlow(x, y) :- Assign(x, z), MemoryAlias(z, y).
```

Structured data requires pre-processing

Java/C++ expression:

```
foo(bar(x), x+y)
```

Tagged s-expression:

```
(app foo
     (app bar x)
     (app + x y))
```

Structured data requires pre-processing

Java/C++ expression:

```
foo(bar(x), x+y)
```

Parsing

Tagged s-expression:

```
(app foo
     (app bar x)
     (app + x y))
```

Pre-processing to flatten nested structure into first-order facts

```
(app e0 foo e1 e2)
(app e1 bar x)
(app e2 + x y)
```

Each (sub)term is assigned a unique identity which from Datalog's point of view extends the Herbrand universe.

Direct Implementation of ADTs in Datalog

```
subexpr($lam(x, eb), eb) :- $lam(x, eb).
```

Direct Implementation of ADTs in Datalog

```
subexpr($lam(x, eb), eb) :- $lam(x, eb).
```

```
subexpr($lam(x, eb), eb) :- subexpr(_, $lam(x, eb)).
```

These $ values are linked structures on the heap, not top-level facts in the Database! **ADTs cannot trigger rule evaluation.**

Direct Implementation of ADTs in Datalog

```
shadows($lam(x, eb0), $lam(x, eb1))
  :-  subexpr(_, $lam(x, eb0)),
      within(eb0, $lam(x, eb1)).
```

The formal parameters x must unify, but this cannot be operationalized via join!

ADTs are not indexed as facts, leading also to poor code generation where a Cartesian product is generated and filtered, as opposed to a proper join as for flat relational data.

Soufflé

Slog: Datalog with first-class facts

All facts are data
(may be *referenced*)

Every datum is a fact
(*triggers* rule evaluation)

Structured data is
automatically indexed and may
be referenced or optimized via
joins and indices

Enables call/return idioms,
defunctionalization, and
(data-parallel) functional programming

Apples-to-apples comparison with  Soufflé

```
// Eval states
ret(v,k) :- eval($Ref(x),env,k,_),
    env_map(x,env,a), store(a,v).
ret($Clo($Lam(x,body),env),k) :-
    eval($Lam(x,body),env,k,_).
eval(ef,env,$ArK(ea,env,$App(ef,ea),c,k),c) :-
    eval($App(ef,ea),env,k,c).
// Ret states
eval(ea,env,$FnK(vf,call,c,k),c) :-
    ret(vf,$ArK(ea,env,call,c,k)).
apply(call,vf,va,k,c) :- ret(va,$FnK(vf,call,c,k)).
ret(v,k) :- ret(v,$KAddr(e,env)),kont_map($KAddr(e,env),k).
program_ret(v) :- ret(v, $Halt()).
// Apply states
eval(body,$UpdateEnv(x,
    $Address(x,[call,[hist0,[hist1,nil]]]),env),
    $KAddr(body,$UpdateEnv(x,
        $Address(x,[call,[hist0,[hist1,nil]]]),env)),
                        [call,[hist0,[hist1,nil]]]),
kont_map($KAddr(body,
    $UpdateEnv(x,$Address(x,
        [call,[hist0,[hist1,nil]]]),env)),k),
store($Address(x,[call,[hist0,[hist1,nil]]]),va),
env_update(env,$UpdateEnv(x,
    $Address(x,[call,[hist0,[hist1,nil]]]),env)) :-
    apply(call,$Clo($Lam(x,body),env),va,k,[hist0,[hist1,_]]).
// Propagate free vars
env_map(x,env1,a) :- env_update(env,env1),
     env1 = $UpdateEnv(x,a,env).
env_map(x,env1,a) :- env_update(env,env1),
     env_map(x,env,a).
```

```
;; Eval states
[(eval (ref x) k c)
 -->
 (ret {store (addr x c)} k)]
[(eval (lam x body) k c)
 -->
 (ret (clo (lam x body) c) k)]
[(eval (app ef ea) k c)
 -->
 (eval ef (ar-k ea (app ef ea) c k) c)]


;; Ret states
[(ret vf (ar-k ea call c k))
 -->
 (eval ea (fn-k vf call c k) c)]
[(ret va (fn-k vf call c k))
 -->
 (apply call vf va k c)]
[(ret v (kaddr e c))
 (store (kaddr e c) k)
 -->
 (ret v k)]
;; Apply states
[(apply call (clo (lam x Eb) _) va k c)
 -->
 (eval Eb (kaddr Eb c') c')
 (store (kaddr Eb c') k)
 (store (addr x c') va)
 (= c' {tick !(do-tick call c)})]
; Propagate free vars
[(free y (lam x body))
 (apply call (clo (lam x body) clam) _ _ c)
 -->
 (store (addr y {tick !(do-tick call c)})
        {store (addr y clam)})]
```

Apples-to-apples comparison with  Soufflé

	Size	Iters	Cf. Pts	Sto. Sz.	8 Processes		64 Processes	
					Slog	Soufflé	Slog	Soufflé
3-k-CFA	8	1,193	98.1k	23.4k	**0:01**	1:07	0:02	00:15
	9	1,312	371.0k	79.9k	**0:02**	14:47	0:03	02:56
	10	1,431	1.44M	291.3k	0:06	🕘	**0:05**	45:49
	11	1,550	5.68M	1.11M	0:27	🕘	**0:16**	🕘
	12	1,669	22.5M	4.32M	2:14	🕘	**1:07**	🕘
	13	1,788	89.8M	17.0M	12:17	🕘	**5:08**	🕘
4-k-CFA	9	1,363	311.8k	65.4k	**0:01**	14:38	0:03	02:08
	10	1,482	1.20M	229.6k	**0:05**	🕘	0:05	40:30
	11	1,601	4.69M	853.5k	0:20	🕘	**0:13**	🕘
	12	1,720	18.6M	3.28M	1:40	🕘	**0:53**	🕘
	13	1,839	73.8M	12.9M	8:44	🕘	**3:58**	🕘
	14	1,958	294.4M	50.9M	1:00:53	🕘	**35:46**	🕘
5-k-CFA	9	1,429	203.7k	50.7k	**0:02**	05:30	0:03	1:15
	10	1,548	756.9k	167.3k	**0:04**	65:20	0:04	15:08
	11	1,667	2.91M	597.5k	0:13	🕘	**0:08**	3:16:06
	12	1,786	11.4M	2.25M	0:56	🕘	**0:27**	🕘
	13	1,905	45.2M	8.69M	4:38	🕘	**2:00**	🕘
	14	2,024	179.9M	34.2M	25:14	🕘	**9:58**	🕘

Apples-to-apples comparison with  Soufflé

	Size	Iters	Cf. Pts	Sto. Sz.	8 Processes		64 Processes	
					Slog	Soufflé	Slog	Soufflé
3-k-CFA	8	1,193	98.1k	23.4k	**0:01**	1:07	0:02	00:15
	9	1,312	371.0k	79.9k	**0:02**	14:47	0:03	02:56
	10	1,431	1.44M	291.3k	0:06	🕘	**0:05**	45:49
	11	1,550	5.68M	1.11M	0:27	🕘	**0:16**	🕘
	12	1,669	22.5M	4.32M	2:14	🕘	**1:07**	🕘
	13	1,788	89.8M	17.0M	12:17	🕘	**5:08**	🕘
4-k-CFA	9	1,363	311.8k	65.4k	**0:01**	14:38	0:03	02:08
	10	1,482	1.20M	229.6k	**0:05**	🕘	0:05	40:30
	11	1,601	4.69M	853.5k	0:20	🕘	**0:13**	🕘
	12	1,720	18.6M	3.28M	1:40	🕘	**0:53**	🕘
	13	1,839	73.8M	12.9M	8:44	🕘	**3:58**	🕘
	14	1,958	294.4M	50.9M	1:00:53	🕘	**35:46**	🕘
5-k-CFA	9	1,429	203.7k	50.7k	**0:02**	05:30	0:03	1:15
	10	1,548	756.9k	167.3k	**0:04**	65:20	0:04	15:08
	11	1,667	2.91M	597.5k	0:13	🕘	**0:08**	3:16:06
	12	1,786	11.4M	2.25M	0:56	🕘	**0:27**	🕘
	13	1,905	45.2M	8.69M	4:38	🕘	**2:00**	🕘
	14	2,024	179.9M	34.2M	25:14	🕘	**9:58**	🕘

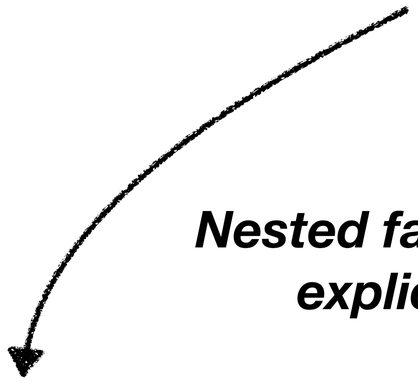
We can generate near-arbitrary speedups for analyses that use structured data because Souffle does not use indices for ADTs, while our paradigm reuses index generation for facts!

Slog: Datalog with first-class facts

```
[(shadows (lam x eb0) (lam x eb1))
 <--
 (lam x eb0)
 (within eb0 (lam x eb1))]
```

Slog: Datalog with first-class facts

```
[(shadows (lam x eb0) (lam x eb1))
 <--
 (lam x eb0)
 (within eb0 (lam x eb1))]
```

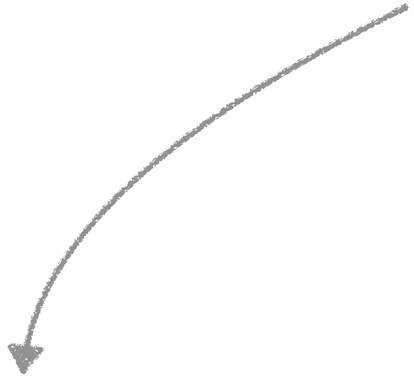


Nested facts desugars to flat facts with explicit id values that link them

```
[(shadows lam0 lam1)
 <--
 (= lam0 (lam x eb0))
 (= _ (within eb0 lam1))
 (= lam1 (lam x eb1))]
```

Slog: Datalog with first-class facts

```
[(shadows (lam x eb0) (lam x eb1))
 <--
 (lam x eb0)
 (within eb0 (lam x eb1))]
```



A ?-clause in the head of a rule is also a body clause

```
[(shadows lam0 lam1)
 <--
 (= lam0 (lam x eb0))
 (= _ (within eb0 lam1))
 (= lam1 (lam x eb1))]
```

```
[(shadows ?(lam x eb0) (lam x eb1))
 <--
 (within eb0 (lam x eb1))]
```

Free Variables

The function terminates because second argument finite (bounded by input program)

```
;; An example:
(λ (param "w")
 (app (λ (param "x") (ref "y"))
      (λ (param "y") (app (ref "z")
                          (ref "y")))))
```

```
(free x ?(ref x))

[(free x ?(app e0 e1))
 <--
 (or (free x e0)
     (free x e1))]

[(free x ?(λ (param a) e0))
 <--
 (=/= x a)
 (free x e0)]
```

Slog's ability to reference / observe facts allows us to define ad-hoc polymorphic functions over structured facts:

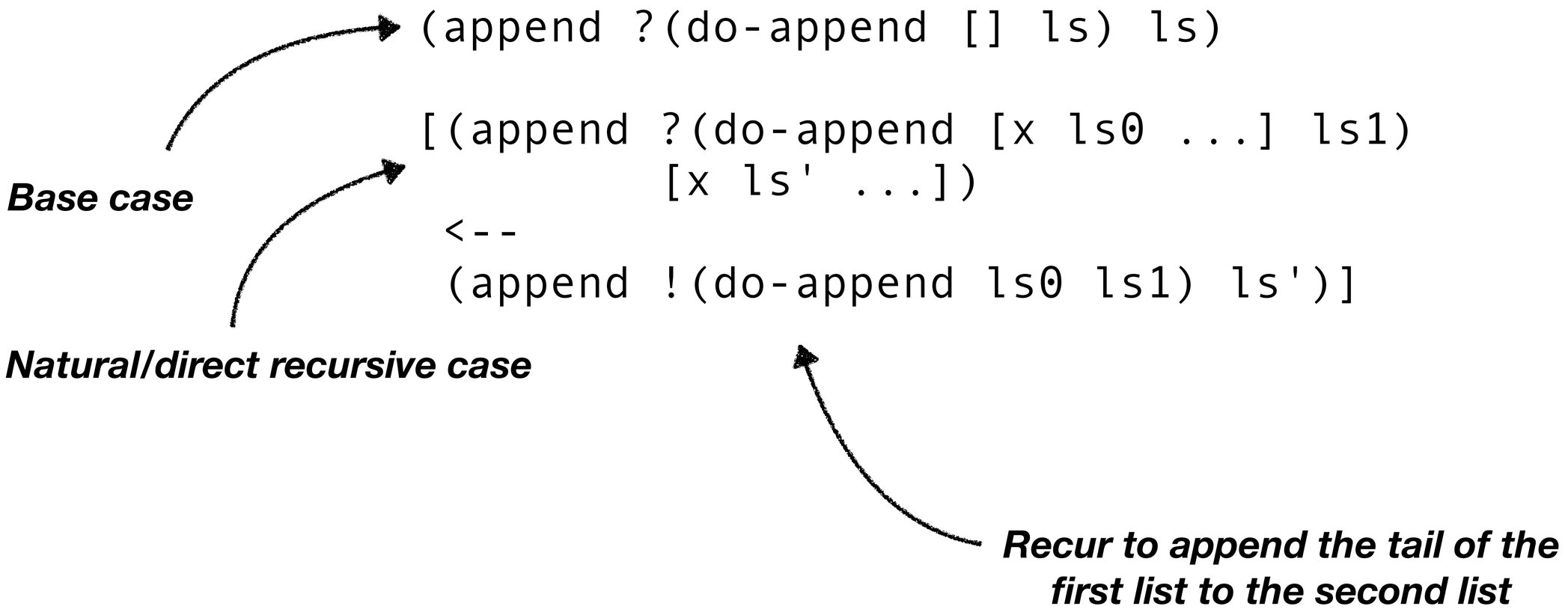
👍 — Anticipates defunctionalization

👍 — Enables elegant transliteration from natural deduction rules (when they are "algorithmic")

👍 — Ubiquitous data parallelism for free throughout computation

👎 — No type system to guarantee totality, type correctness, termination, finiteness, etc.

A demand transform for natural deduction



A demand transform for natural deduction

```
(append ?(do-append [] ls) ls)

[(append ?(do-append [x ls0 ...] ls1)
         [x ls' ...])
 <--
 (append !(do-append ls0 ls1) ls')]
```

Desugaring a ! clause in the body of a rule splits the rule into a rule for the intermediate head and a continuation that finishes the rule.

```
[(do-append ls0 ls1)
 <--
 (do-append [x ls0 ...] ls1)]
```

```
[(append call [x ls' ...])
 <--
 (append (do-append ls0 ls1) ls')
 (= call (do-append [x ls0 ...] ls1))]
```

A natural-deduction-style λ-calculus interpreter

```
; Var
[(env-map env x val)
 -->
 (interp ?(clo (ref x) env) val)]
```

$$\boxed{e, \rho \Downarrow v}$$

$$\textsc{Var}\quad \frac{v = \rho(x)}{x, \rho \Downarrow v}$$

A natural-deduction-style λ-calculus interpreter

```
; Var
[(env-map env x val)
 -->
 (interp ?(clo (ref x) env) val)]
; Abs
(interp ?(clo (lam x Eb) env)
        (clo (lam x Eb) env))
```

$$\boxed{e, \rho \Downarrow v}$$

$$\textsc{Var}\quad \frac{v = \rho(x)}{x, \rho \Downarrow v}$$

$$\textsc{Abs}\quad \frac{}{(\lambda\,(x)\,e), \rho \Downarrow \langle (\lambda\,(x)\,e), \rho \rangle}$$

A natural-deduction-style λ-calculus interpreter

```
; Var
[(env-map env x val)
 -->
 (interp ?(clo (ref x) env) val)]
; Abs
(interp ?(clo (lam x Eb) env)
        (clo (lam x Eb) env))
; App
[(interp !(clo Ef env) (clo (lam x Eb) env'))
 (interp !(clo Ea env) Eav)
 (interp !(clo Eb (ext-env env' x Eav)) v)
 -->
 (interp ?(clo (app Ef Ea) env) v)]
```

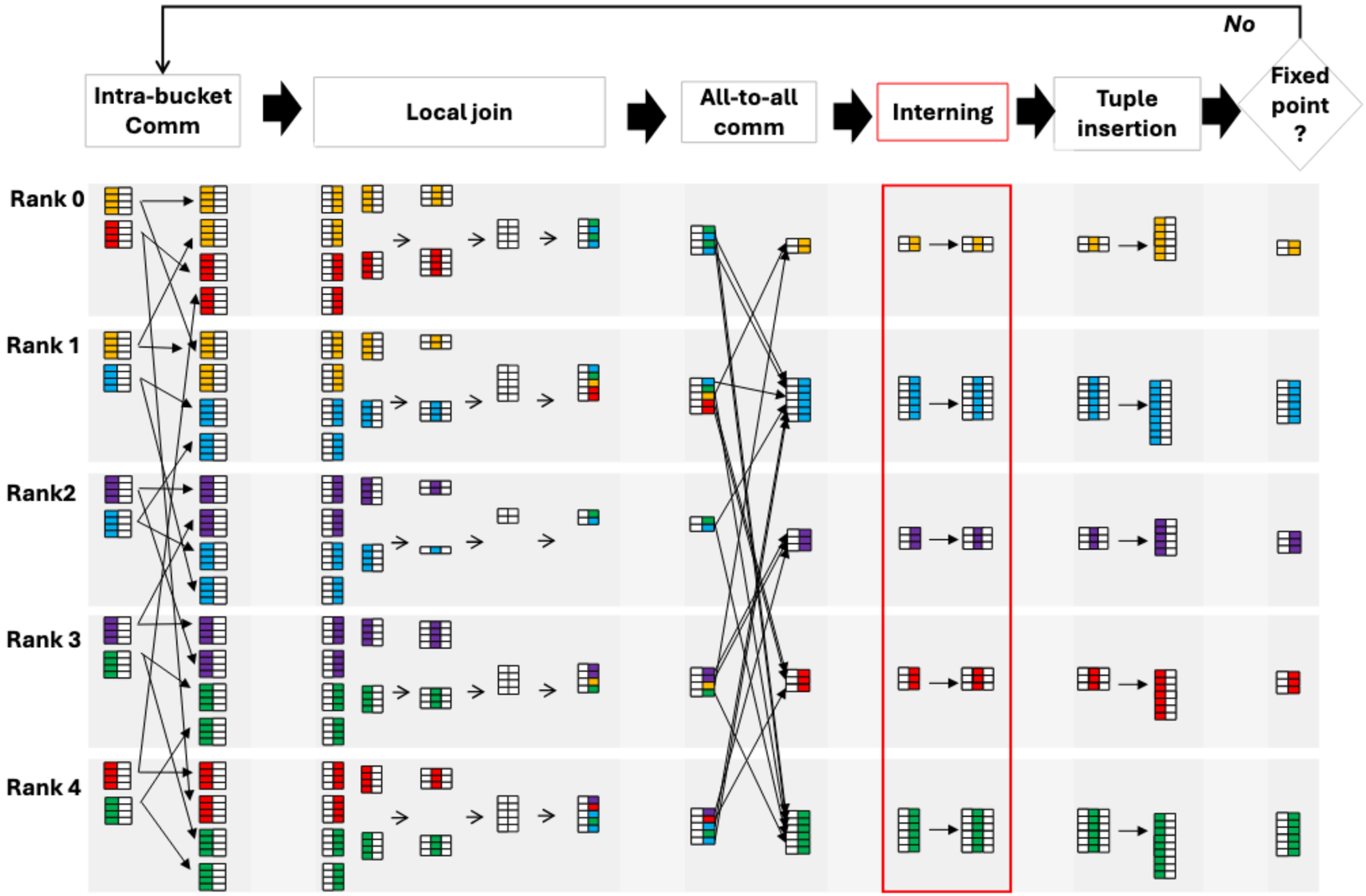
$\boxed{e, \rho \Downarrow v}$

$$\textsc{Var}\quad \frac{v = \rho(x)}{x,\rho \Downarrow v}$$

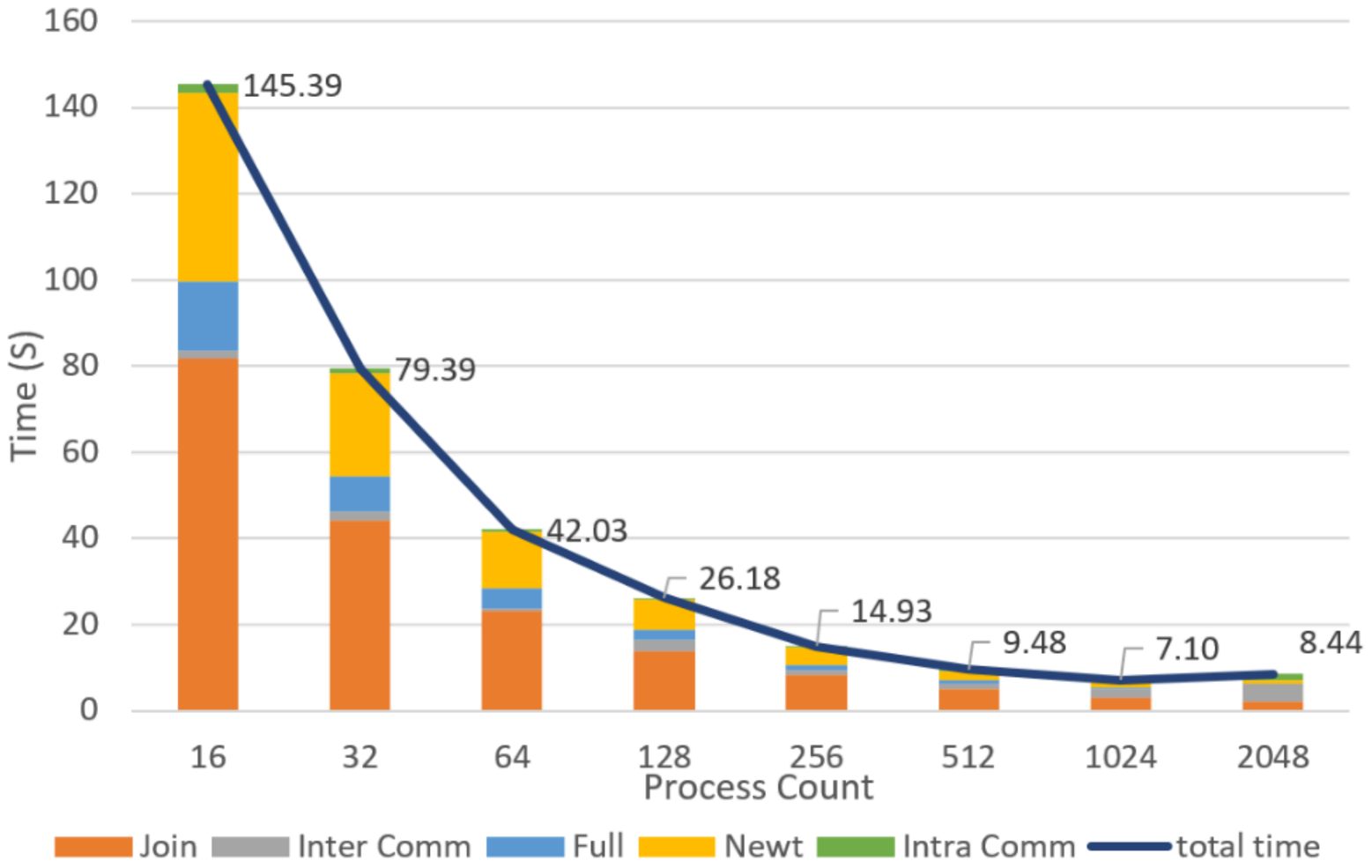
$$\textsc{Abs}\quad \frac{}{(\lambda\,(x)\,e),\rho \Downarrow \langle(\lambda\,(x)\,e),\rho\rangle}$$

$$\textsc{App}\quad \frac{e_f,\rho \Downarrow \langle(\lambda\,(x)\,e_b),\rho_\lambda\rangle \qquad e_a,\rho \Downarrow v_a \qquad e_b,\rho_\lambda[x \mapsto v_a] \Downarrow v}{(e_f\ e_a),\rho \Downarrow v}$$

Data-parallel communication-avoiding implementation



Scaling a pointer analysis of the linux kernel



Apples-to-apples why-provenance comparison

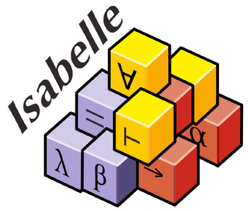
Query	Dataset	Nemo (Rust)	Vlog (Java)	RDFox (threads) 1	RDFox (threads) 12	SLOG (threads) 1	SLOG (threads) 12
Galen	15	1.63	7.38	0.20	0.14	0.68	**0.13**
	25	2.35	17.7	0.33	0.19	0.92	**0.17**
	50	34.5	415	2.30	**1.98**	12.0	2.19
CSDA	httpd	72.2	152	57.9	40.7	127	**22**
	linux	548	🕒	315	230	6,237	**116**
	postgres	234	🕒	138	95.2	320	**62.6**
Andersen	10,000	1.38	3.94	1.08	0.61	1.21	**0.19**
	50,000	7.70	22.6	6.43	3.04	7.84	**0.94**
	100,000	16.5	42.3	13.7	6.78	21.7	**2.28**
	500,000	119	386	78.8	30.4	144	**14.6**
TC	SF.cedge	439	296	443	291	628	**183**
	wiki-vote	235	1372	391	337	485	**91**
	Gnutella04	376	🕒	441	321	725	**124**

Formal perspectives on Slog: DL_S, $DL^{\exists!}$

- Our paper offers some formal perspectives on Slog; we provide:
  - A variant of the restricted chase.
  - A model-theoretic semantics based on a core language $DL^{\exists!}$

$$\left(\exists! id_H.\ id_H = H(x, y, \ldots)\right) \leftarrow id_B = B(a, b, \ldots) \wedge \ldots$$

  - A fixed-point semantics based on a core language DL_S
- We formalize key properties and an equivalence in in Isabelle/HOL

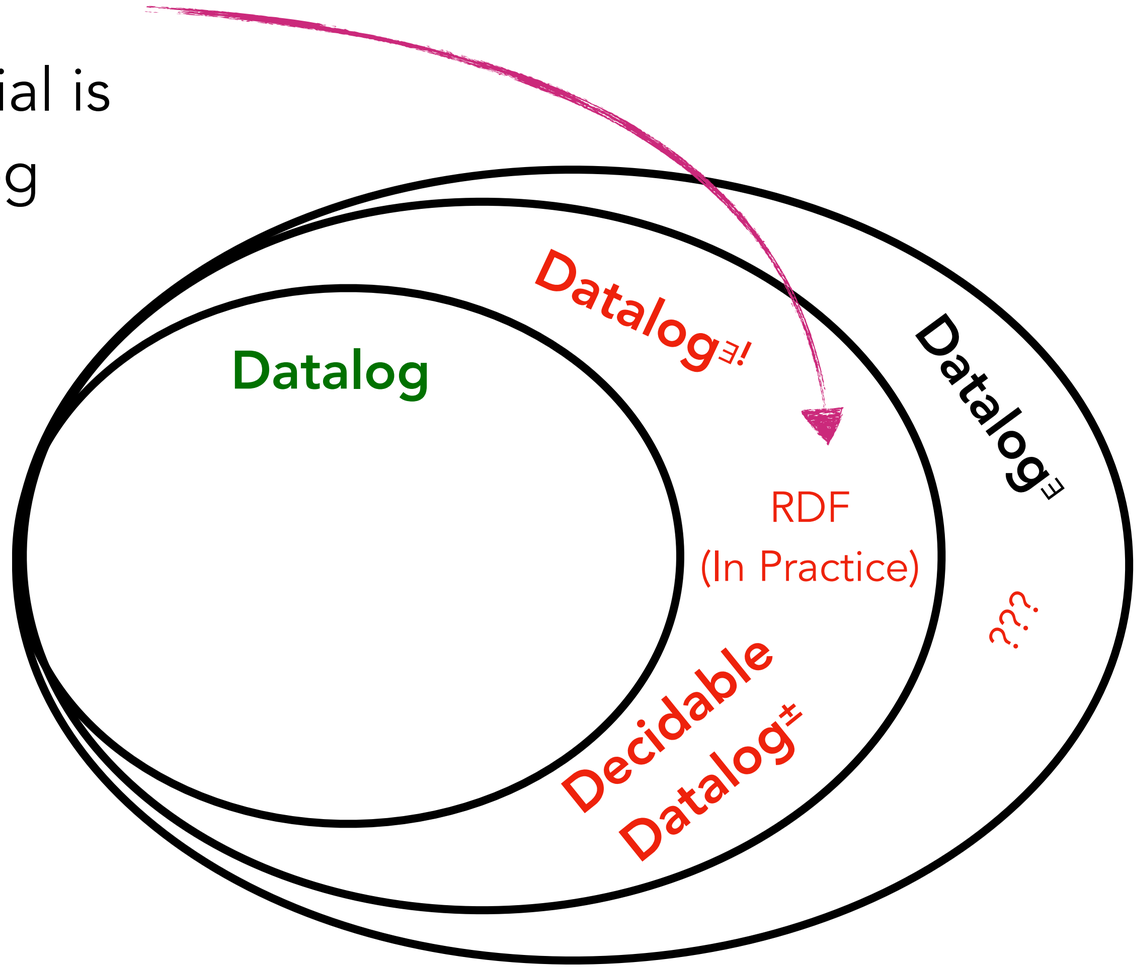


Datalog$^{\exists}$

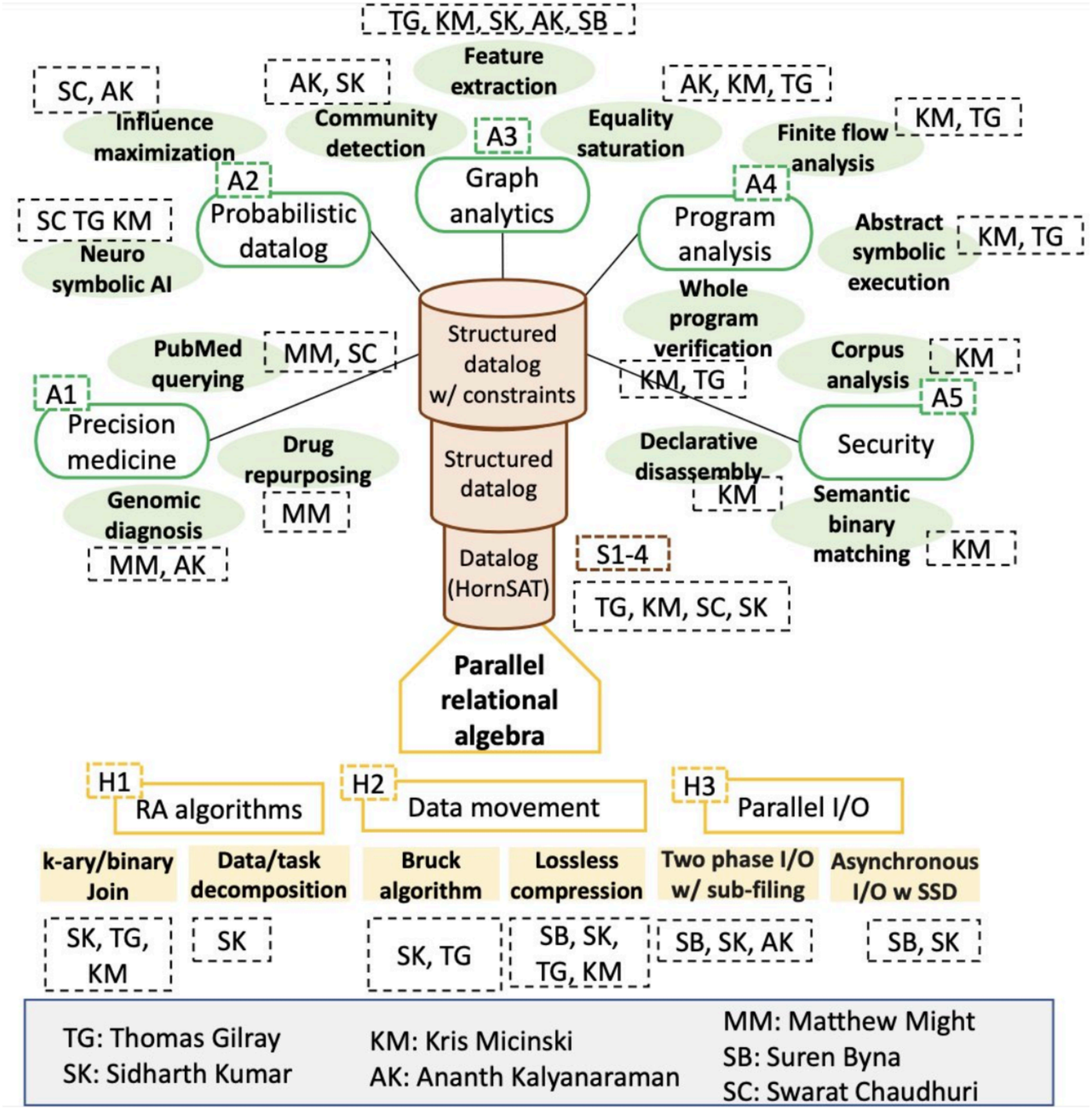
- Extends Datalog to existential quantifiers in rule heads
- `∃teacher. teaches(teacher,class) ← enrolled(student, class).`
- Popular extension: reasoning over ontologies, knowledge graphs, RDF workloads, etc…
- However, full Datalog^\exists is undecidable in general:
- `∃z. edge(y,z) ← edge(x,y).`
- In practice, all engines implement (often semi-decidable) subsets, often from a family of languages called Datalog$^{\pm}$ with various restrictions: Shy, Warded, weak-acylicity, etc.

We argue that RDF workloads are **in practice** around here (not proven, our speculation!)

Datalog with an unrestricted existential is fundamentally a disjunctive Datalog



Huge thanks to NSF (PPoSS planning / LARGE) ❤️❤️❤️



Thank You!

- Highest-performance Datalog engines to date (we believe!)
- Our current work is to harmonize three threads:
  - automated defunctionalization of functional code into DL
  - Slog / DL^E!
  - our GPU backends

harp-lab.com